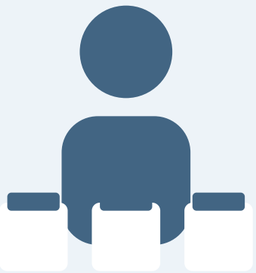
COURSE OVERVIEW

Learn programming. Help a team plan pickups.

Three groups need to work together

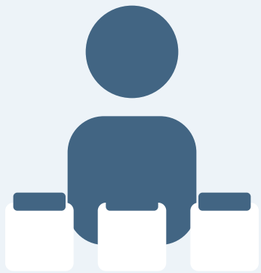
Food business



“We have food to offer.”

Tells the team what is available.

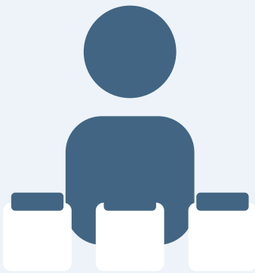
University volunteers



“Who can collect it?”

Plan the pickup with a coordinator.

Community partner



“Has the food arrived?”

Confirms the handoff.

Our fictional university team helps food businesses coordinate pickups with community partners.

Why messages alone can be confusing

Separate messages

“Four boxes.” What is inside?

“I will collect.” Which pickup?

“Done.” Collected or delivered?

A shared pickup record

WHAT	Bread · 4 containers
WHO	Named volunteer team
STATUS	Planned / collected / delivered

An information system helps people keep agreed information and follow the same steps.

What does programming add?

1

Information we enter

4 containers
3 planned portions in each

2

Instructions we write

Multiply the two counts.
Print a clear label.

3

An answer we can use

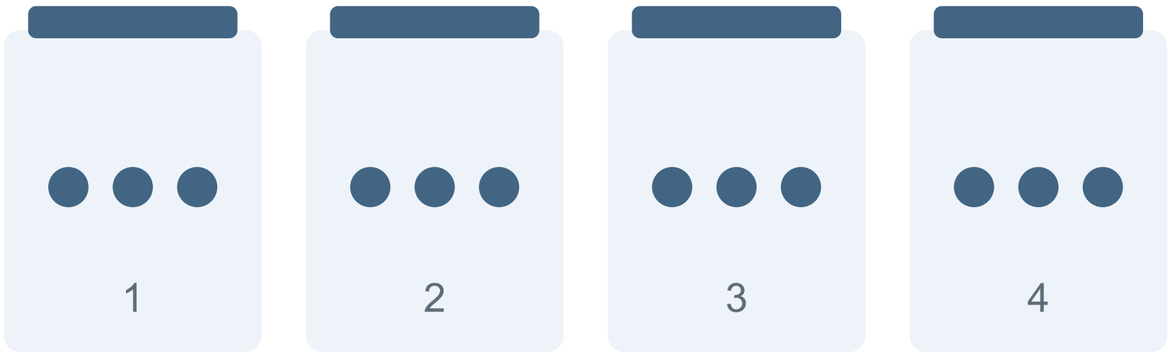
12 planned portions
The team can discuss the plan.

A program is a set of instructions a computer follows.

Start with one useful answer. Add more instructions as the course progresses.

Your first program starts with a small calculation

4 containers



3 declared portions per container

3 × 4 = 12

Planned portions

A calculation from declared inputs.

4 containers × 3 declared portions each = 12 planned portions.

Follow one pickup from plan to delivery

The same offer: 4 bread containers are available; each represents 3 planned portions.

1

Plan

Choose 3 containers. 3 × 3 = 9 planned portions.



2

Set aside

Accept the booking. 1 container remains available.



3

Collect

A volunteer picks up the same 3 containers.



4

Deliver

The partner receives the same 3 containers.



The same 3 containers change status. A booking is not yet a collection or delivery.

What will you do on your computer?

A SMALL EXAMPLE IN YOUR NOTEBOOK

```
containers = 4
portions_each = 3
planned = containers * portions_each
print("Planned portions:", planned)
```

```
Planned portions: 12
```

Your work

1. Predict the answer.
2. Run the example.
3. Change 4 to 3.
4. Explain what happened.

A notebook puts explanations, editable code and your observations in one place.

People make decisions. Programs help with records.

People

Clarify what is offered.
Agree who will collect it.
Approve arrangements and confirm delivery.

The program

Calculate a plan.
Apply an agreed rule.
Save information and show what still needs attention.

You, the learner

Turn one small need into instructions.
Try examples.
Check and explain the result.

People approve arrangements, collect food and confirm what happened.

Ten weeks: start small, then connect the parts

1

Weeks 1–4

Calculate and choose
Build small Python rules.
Check examples and explain them.

2

Weeks 5–8

Remember and coordinate
Check incoming information.
Save plans and track pickups.

3

Weeks 9–10

Improve and share
Change one useful feature.
Let a classmate run it.

Every week supplies a starting point; you can join even if last week’s notebook is unfinished.

Week 01 · A first planning note

WEEK 01

A first planning note

The coordinator asks: how much are we planning?

PICTURE ONE EXAMPLE

4 containers × 3 = 12 planned portions.

PYTHON IDEAS

Variables · types · arithmetic · strings · .py execution

YOU WILL SHOW

Inputs, labelled total and one question about units.

NEXT USE

Week 2 uses a total in a review-first decision.

One small need → your Python work → a result you can check.

Week 02 · Choose a coordination label

WEEK 02

Choose a coordination label

The coordinator needs a rule for when a request can proceed.

PICTURE ONE EXAMPLE

Not reviewed yet → wait for the coordinator.

PYTHON IDEAS

Comparisons · Boolean logic · if/elif/else · boundary tests

YOU WILL SHOW

A six-row decision table and checked labels.

NEXT USE

Week 3 calculates the total from several batch lines.

One small need → your Python work → a result you can check.

Week 03 · Add several food items

WEEK 03

Add several food items

A pickup can contain more than one kind of food.

PICTURE ONE EXAMPLE

Bread: 6 portions + rice: 10 → total 16.

PYTHON IDEAS

Lists · dictionaries · for loops · accumulators

YOU WILL SHOW

Three labelled totals, a hand trace and empty/split tests.

NEXT USE

Week 4 puts a reusable rule behind a function boundary.

One small need → your Python work → a result you can check.

Week 04 · Reuse a rule and test it

WEEK 04

Reuse a rule and test it

The same calculation should work with different inputs.

PICTURE ONE EXAMPLE

New inputs → the same function → a summary.

PYTHON IDEAS

Functions · return · main() · imports · assertions · debugging

YOU WILL SHOW

A callable rule, boundary checks and an individual variant.

NEXT USE

Week 5 adds checked external inputs to reusable logic.

One small need → your Python work → a result you can check.

Week 05 · Check incoming information

WEEK 05

Check incoming information

Messages and table rows may contain missing or unclear counts.

PICTURE ONE EXAMPLE

Missing count → ask; do not guess.

PYTHON IDEAS

CSV · conversion · validation · exceptions · CLI arguments

YOU WILL SHOW

A disposition, reason and source identity for every input row.

NEXT USE

Week 6 saves identifiable records and historical units.

One small need → your Python work → a result you can check.

Week 06 · Keep a plan after closing

WEEK 06

Keep a plan after closing

The next volunteer needs to find the plan tomorrow.

PICTURE ONE EXAMPLE

Save → close → reopen → find the same plan.

PYTHON IDEAS

Objects/methods · SQLite · IDs · relationships · parameters

YOU WILL SHOW

A saved plan, line-unit snapshots and fresh-connection lookup.

NEXT USE

Week 7 groups operational record changes in one transaction.

One small need → your Python work → a result you can check.

Week 07 · Book a complete pickup

WEEK 07

Book a complete pickup

A booking must not leave only part of a request set aside.

PICTURE ONE EXAMPLE

One item unavailable → all counts stay unchanged.

PYTHON IDEAS

Transactions · guarded UPDATE · exceptions · snapshots · tests

YOU WILL SHOW

Complete success, or unchanged state after a required failure.

NEXT USE

Week 8 reports accepted reservations and explicit events.

One small need → your Python work → a result you can check.

Week 08 · See what still needs attention

WEEK 08

See what still needs attention

The coordinator needs to distinguish promises from actual events.

PICTURE ONE EXAMPLE

Collected, but not delivered → follow up.

PYTHON IDEAS

SQL aggregation · reconciliation · HTTP · JSON · validation

YOU WILL SHOW

Stage evidence, six intake outcomes and a project proposal.

NEXT USE

Week 9 implements one need with two agreed acceptance cases.

One small need → your Python work → a result you can check.

Week 09 · Improve one useful feature

WEEK 09

Improve one useful feature

Choose one small problem a volunteer actually faces.

PICTURE ONE EXAMPLE

Example: make pending pickups easier to find.

PYTHON IDEAS

Scoped functions/queries · acceptance · regression · peer review

YOU WILL SHOW

A working change, peer finding, revision and retest.

NEXT USE

Week 10 makes the feature reproducible by another operator.

One small need → your Python work → a result you can check.

Week 10 · Let someone else use your work

WEEK 10

Let someone else use your work

A classmate needs to run the feature using your guide.

PICTURE ONE EXAMPLE

Your classmate runs it and explains the result.

PYTHON IDEAS

Clean starts · run commands · recovery · independent verification

YOU WILL SHOW

A reproducible guide, operator demo and individual explanation.

NEXT USE

The next operator can trace the result and its limitations.

One small need → your Python work → a result you can check.

Every week: learn Python, practise, then build

Session 1 · Python

Start with the programming topic.
Read syntax and trace short code.
Use the matching Basics reference.

Session 2 · Practice

Work in Python Basics.
Predict, run and change examples.
Explain a result with a partner.

Session 3 · Project

Use a short design briefing.
Implement the weekly component.
Discuss, check and keep evidence.

Three 90-minute sessions per week; most project time belongs to your work.

Use the right material for the job

Python Basics

19 reference sections
Weekly practice hubs
Examples and sample answers

Weekly notebook

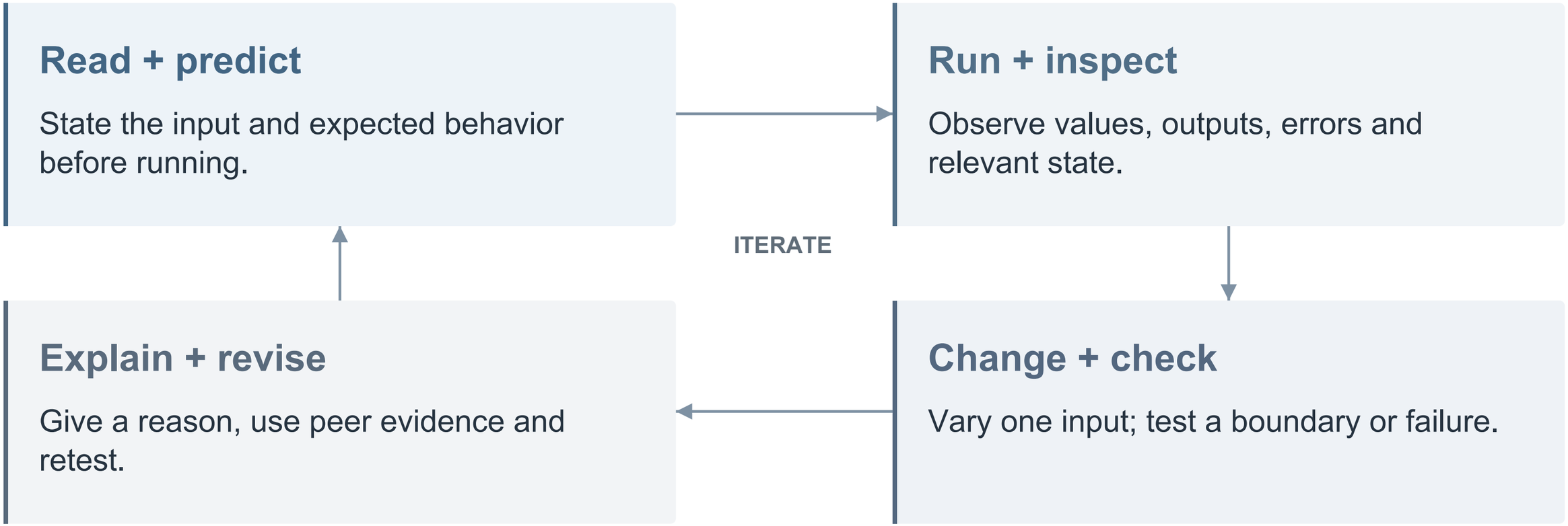
Project context and diagrams
Supplied data and scaffolds
Your implementation and evidence

Assignment brief

Detailed task and expected evidence
A small example
Midterm/final guides separately

Each week has a starting point; an unfinished earlier notebook does not block you.

Make learning visible in a small cycle



Save the required evidence and make sure you can explain it; optional extensions can wait.

Your final project: one small, verified improvement

Agree a need

Pitch in Week 8.
Choose a stakeholder and scope.
Freeze two acceptance cases.

Build and verify

Use the supplied baseline.
Keep a regression/failure case.
Record a peer finding and retest.

Demonstrate and hand over

Week 10: let a peer operate.
Provide reset/run instructions.
Explain your work individually.

A reliable Python or SQL change is enough; a web app or paid AI integration is not required.

Six short research tasks support the project

Research / topic	Assigned	Shared
1 · Goals + systems	Week 1	Week 2
2 · Functions + tests	Week 2	Week 3
3 · Input validation	After Week 4 midterm	Week 5
4 · Persistent records	Week 5	Week 6
5 · Failure handling	Week 6	Week 7
6 · Output checks	Week 7	Start of Week 8

Proposed 5% each · sources, example and your reasoning · presentation is not required.

Assessment rewards building, checking and explaining

Component	Proposed weight	Evidence
Participation	10%	Attendance and reflection
Six research tasks	30%	Sources, example, reasoning
Individual midterm	30%	Function, tests, ZIP, explanation
Final project	30%	Change, verification, handoff

The instructor and university will confirm the plan before grading begins.

Get started and keep questions useful

Open the package

Use learning_materials/ and keep its support folders together.
Follow the setup guide.

Choose an environment

Local Python/Jupyter or Colab.
Upload the support ZIP in Colab.
Save your work and a backup.

Ask with evidence

Week + small input + expectation
+ observed result + question.
Use the announced class route.

Use the course home for available weeks, links, assignment briefs and setup instructions.

Further reading in English

1. ## Python Tutorial — classes and exceptions

   https://docs.python.org/3/tutorial/

2. ## Python sqlite3 — database connections and transactions

   https://docs.python.org/3/library/sqlite3.html

3. ## Python unittest — organizing and running tests

   https://docs.python.org/3/library/unittest.html

Read one relevant section, then bring an example and a question.